



# LISP / SCHEME



# HISTORIA

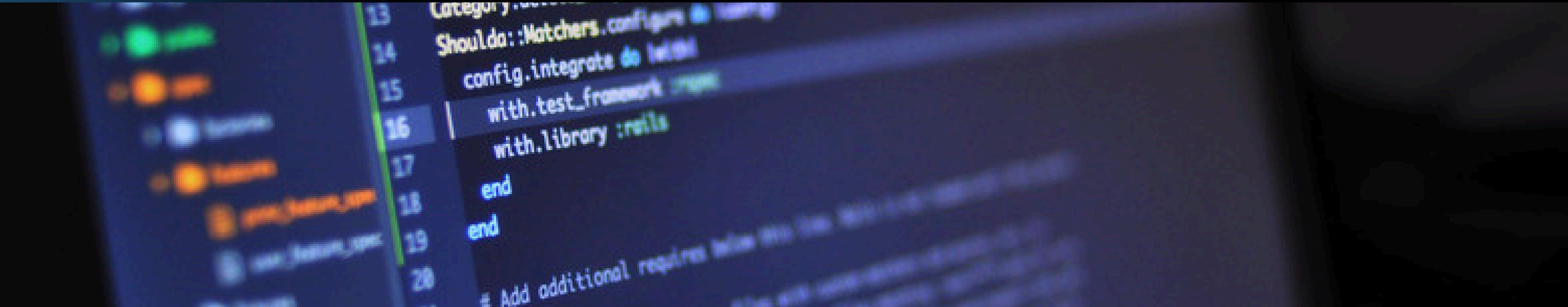
**Lisp (1958):** Creado por John McCarthy. Pionero en inteligencia artificial y computación simbólica.

**Scheme (1975):** Creado por Sussman y Steele. Enfoque minimalista orientado a la educación e investigación.

## CONTEXTO HISTORICO

Estos lenguajes introdujeron el cálculo lambda y la homoiconicidad, sentando las bases de la programación funcional y los sistemas expertos.





# CÓDIGO COMO DATOS

- En Lisp, el código se escribe como listas
- La función va primero: (función dato1 dato2)
- Ejemplo: (+ 2 3) significa "suma 2 y 3"
- Esta idea se llama homoiconicidad



MATEMÁTICA NORMAL

$2+3$

LISP/SCHEME

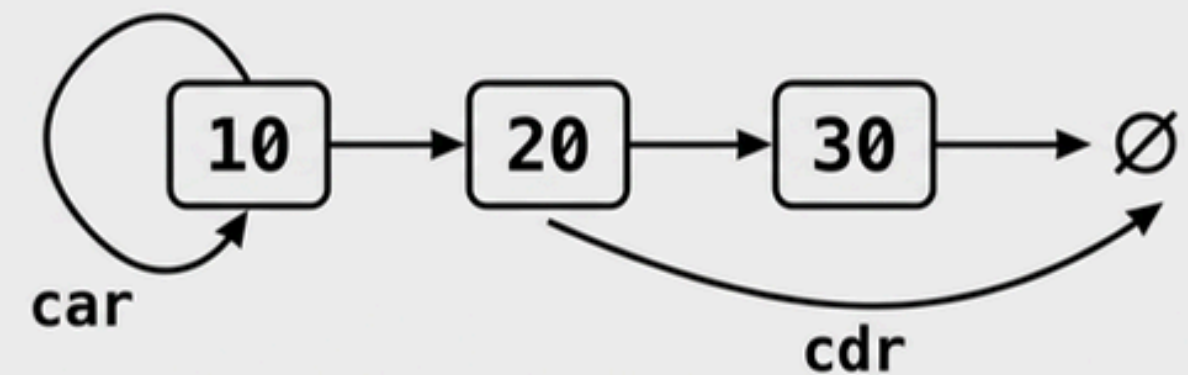
(+ 2 3)



# OPERACIONES BÁSICAS SOBRE LISTAS

- cons → agrega un elemento a la lista
- car → da el primer elemento
- cdr → da el resto de la lista

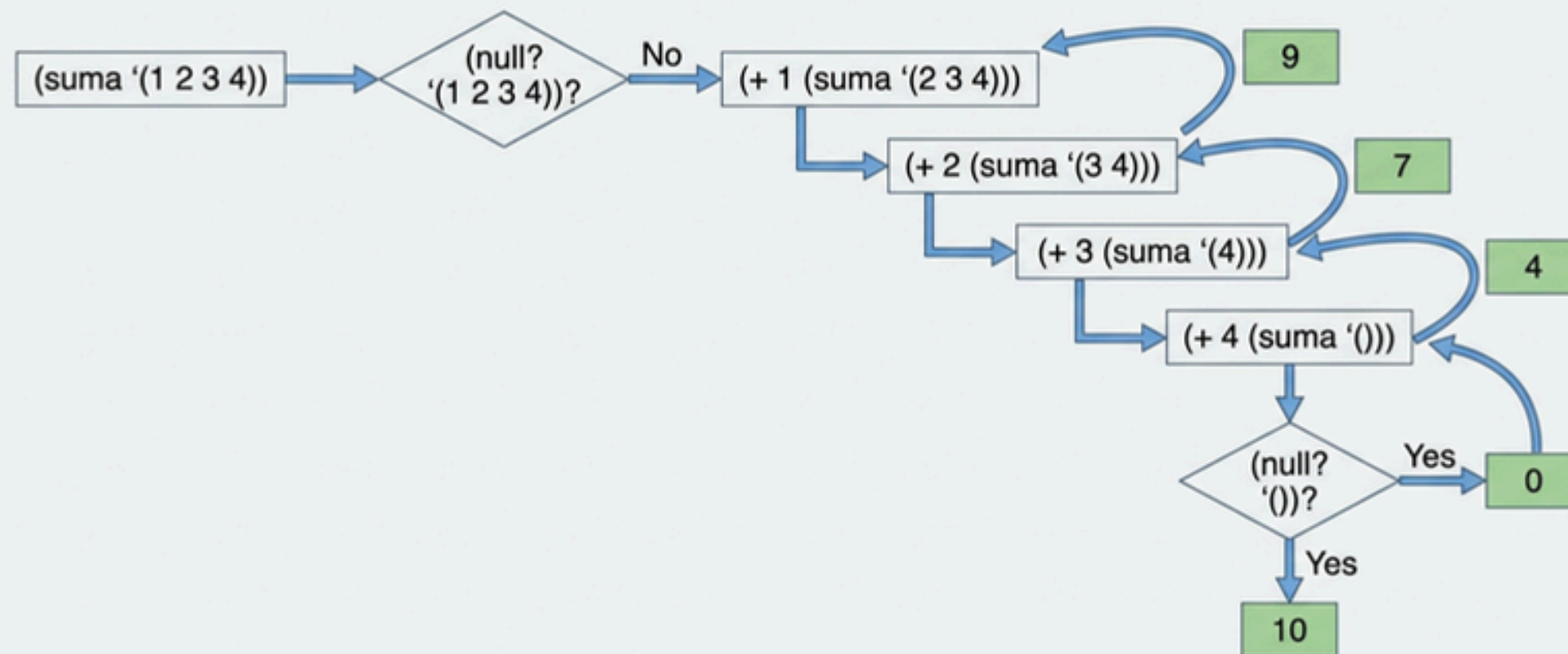
```
user@host:~$ (define numeros '(10 20 30))
user@host:~$ (car numeros)      ; 10
user@host:~$ (cdr numeros)      ; (20 30)
user@host:~$ (cons 5 numeros)   ; (5 10 20 30)
user@host:~$ _
```





# EVALUACIÓN Y RECURSION

```
(define (suma lista)
  (if (null? lista) 0
      (+ (car lista) (suma (cdr lista)))))
```



- Scheme calcula primero los valores, luego ejecuta la función (evaluación estricta)
- Para retrasar un cálculo: delay y force
- Para repetir algo, se usa recursión (una función se llama a sí misma)



# SISTEMA DE TIPOS

**Tipado Estático:** Un tipado estático es aquel que compilado comprueba los tipos de variables y parametros en el tiempo de compilación

**Tipado Dinámico:** El tipado dinámico es el cual comprueba los tipos de variables, parametros y valores en tiempo de ejecución.





# RUNTIME

Runtime es el momento en el que un programa ya esta ejecutandose

- **Compile time:** Antes de ejecutar el programa
- **Runtime:** Mientras el programa está funcionando

# EJEMPLO TIPADO DINAMICO

```
(define x 5)      ; x vale 5  
(display x)      ; 5
```

```
(set! x "hola")   ; ahora x vale "hola"  
(display x)      ; hola
```

La variable x no tiene un tipo fijo



## ⇒ **DEMO EN VIVO**

- **Herramienta: Racket REPL**
- **Ejemplo 1: Imprimir rango (Hola Paradigma).**
- **Ejemplo 2: Suma recursiva de listas usando car y cdr.**



# CASO DE USO REAL



- IA Simbólica: Lisp fue el lenguaje estándar de la Inteligencia Artificial durante décadas, usado para crear sistemas expertos como MYCIN o SHRDLU.
- Educación en el MIT: Scheme fue el lenguaje base de Structure and Interpretation of Computer Programs (SICP), uno de los libros más influyentes en ciencias de la computación.
- Legado actual: Sus conceptos viven hoy en lenguajes modernos como Clojure, Haskell y Elixir.
- (Importante: En la esquina inferior de esta diapositiva debe ir la cita bibliográfica IEEE de SICP o de Lisp para cumplir tu rúbrica).



# BIBLIOGRAFÍAS

- [1] J. McCarthy, "Recursive functions of symbolic expressions and their computation by machine, Part I," Communications of the ACM, vol. 3, no. 4, pp. 184–195, Apr. 1960.
- [2] H. Abelson, G. J. Sussman, and J. Sussman, Structure and Interpretation of Computer Programs, 2nd ed. Cambridge, MA, USA: MIT Press, 1996.
- [3] H. Abelson, G. J. Sussman, and J. Sussman, Structure and Interpretation of Computer Programs, JavaScript ed. Cambridge, MA, USA: MIT Press, 2022.
- [4] G. J. Sussman and G. L. Steele Jr., "Scheme: An interpreter for extended lambda calculus," MIT AI Lab, Cambridge, MA, USA, AI Memo 349, Dec. 1975.
- [5] G. J. Sussman and G. L. Steele Jr., "Scheme: An interpreter for extended lambda calculus," Higher-Order and Symbolic Computation, vol. 11, no. 4, pp. 405–439, Dec. 1998.
- [6] W. Sack, "A matter of interpretation: A review of Structure and Interpretation of Computer Programs (JavaScript edition)," Computational Culture, no. 9, 2023.



**GRACIAS**